

COMP2001: ARTIFICIAL INTELLIGENCE

METHODS – LAB 3 EXERCISES

RECOMMENDED TIMESCALES

This lab exercise is designed to take no longer than two hours, and it is recommended that you complete this within a week. The contents of this exercise were covered in lecture 3.

GETTING SUPPORT

The computing exercises have been designed to facilitate flexible studying and can be worked through in your own time or during timetabled lab hours. It is recommended that you attend the labs, even if you have completed the exercises in your own time, to discuss your solutions and understanding with one of the lab support team and/or your peers. It is entirely possible that you might make a mistake in your solution which leads to a false observation and understanding.

OBJECTIVES

The purpose of this lab exercise is to gain experience in implementing a single point-based (local search) metaheuristic that we covered in lecture 3 and understanding the issues/complexities of the sensitivities of parameter settings.

- COMP2001 framework background including use of backup solutions.
- Implementation of Iterated Local Search (ILS).
- Completion of formative exercises to include tuning of the strengths of the mutation and local search operators.

Warning - The implementation of ILS has parameterised local search and mutation heuristics and these heuristics are re-used from completion of previous lab exercises; ensure that you have completed lab exercises 0 and 1 prior to this exercise.

LEARNING OUTCOMES

By completing this lab exercise, you should meet the following learning outcomes:

- Students should gain experience implementing single point-based metaheuristics for solving combinatorial optimisation problems.
- Students should understand the parameter setting issues and the impact that poor parameter choices have on the performance of such metaheuristics.
- Students should be able to perform statistical tests to compare the performance of variants of the Iterated Local Search algorithm for solving specific instances of the MAX-SAT problem.

TASK 1 – IMPORTING LAB EXERCISE FILES

Download the source code for this lab exercise from the Moodle page. You should find various files related to the metaheuristics, the exercise runner configurations, and the exercise runners themselves.

Drag the **metaheuristics** folder into your IDE so that it is a direct subfolder of "com.aim". Notice that each metaheuristic is in its own package and these packages are in a parent "singlepoint" package; the focus of this exercise is on single point-based metaheuristics.

The "Lab3ExercisesTestFrameConfig" and "Lab3ExercisesRunner" classes should be placed within the "runners" package alongside the ones from labs 1 and 2.

TASK 2 – READING: FRAMEWORK BACKGROUND

For a single point-based search method, the COMP2001 Framework maintains two solutions in memory:

- The solution that we are operating upon (the current solution), and
- A backup of the solution that we started the current iteration with (the backup solution) so that we can reuse it later if we "reject" any modification(s) made to the current solution.

When a `SATHeuristic` is invoked on a problem object with `iSolutionIndex` set to `CURRENT_SOLUTION_INDEX`, it modifies the solution in the current solution index leading to the candidate solution s' being stored in the `CURRENT_SOLUTION_INDEX` and does not modify the backup solution in the `BACKUP_SOLUTION_INDEX` within the intermediate solution memory state. The move should then either be accepted ($s_{i+1} \leftarrow s'_i$) or rejected ($s_{i+1} \leftarrow s_i$) using the respective move acceptance criterion.

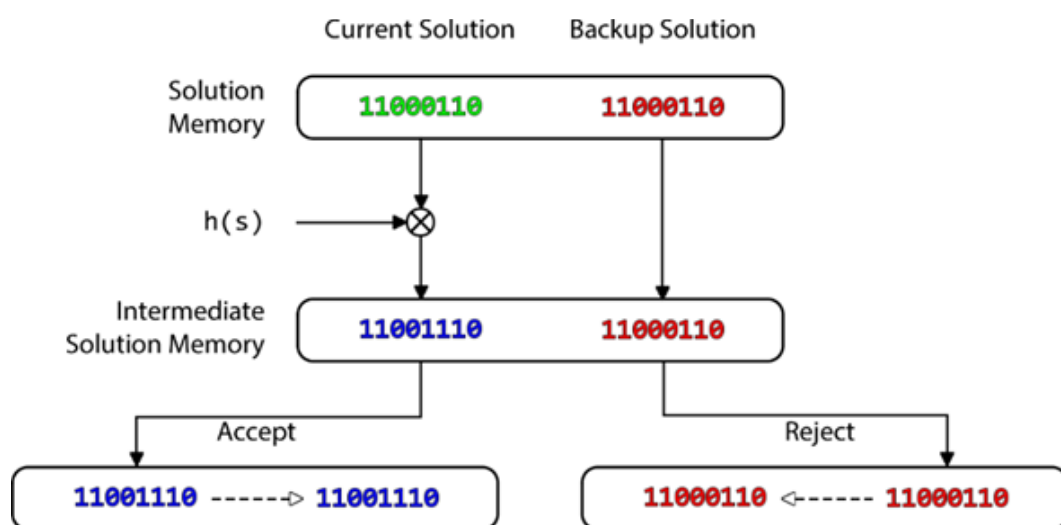


Figure 1 - Single Point-based Search in the COMP2001 Framework. The dashed arrow represents a solution being copied between solution memory indices using the `copySolution(int source_memory_index, int destination_memory_index)` method of the SAT problem.

TASK 3 – ITERATED LOCAL SEARCH [IMPLEMENTATION]

Iterated Local Search (ILS) was covered in lecture 3. Below is the pseudocode from the lecture, adapted to embed intensity of mutation (IOM) and depth of search (DOS) parameters controlling the mutation strength and local search strength respectively. **Remember that the test framework handles solution initialisation and the outer loop of the search method;** hence, only the lines within the REPEAT-UNTIL block need to be implemented in the `runMainLoop()` method.

You will need to think about what the line `s <- s'` really means, and how the framework is designed in order to correctly implement this step (HINT: the order of instructions in the pseudocode and when implemented in the COMP2001 framework is not the same).

For the acceptance criterion, you should accept all non-worsening moves.

Below is the pseudocode for Iterated Local Search embedding the parameters IOM and DOS:

```
s_0 = generatedInitialSolution()           // handled by the lab exercise runner.
s = localSearch(s_0)                       // optional step, not used.
REPEAT                                     // handled by the lab exercise runner.
    s' <- s

    REPEAT iom TIMES:                     // apply iom times mutation.
        s' <- mutate(s')

    REPEAT dos TIMES:                     // apply dos times local search.
        s' <- localSearch(s')

    s <- acceptance(s, s', memory)         // decide which solution to keep as the
                                           // incumbent solution.
UNTIL( termination conditions are met )    // outer loop handled by framework.
return s*                                  // handled by framework.
```

IMPORTANT: Note that there are `SATHeuristic` objects initialised as fields in the `IteratedLocalSearch` class; these contain the mutation and local search low-level heuristics and reuse the implementations of `RandomBitFlipHeuristic` from lab exercise 1, and `DavissBitHC` from lab exercise 2. Ensure that these implementations are completed (and correct; ask us in the labs!) before comparing your results with the expected outputs. Additionally, ensure that `DavissBitHillClimbing` is implemented to accept all **non-worsening moves**.

TASK 4 – FORMATIVE ASSESSMENT (EXPERIMENTS AND MOODLE QUIZ)

The questions in the [Lab 3 Formative Assessment \(link here\)](#) require you to perform a number of experiments using your implementation of Iterated Local Search with various configurations.

Within the test frame configuration, there are two parameters that you should change:

1. Depth of search (`iDepthOfSearch`)
2. Intensity of mutation (`iIntensityOfMutation`)

You can change these settings by modifying them within the `Lab3TestFrameConfig` Class. You should investigate the effects of changing these settings where each setting should be from `{0,1,2,3}`. As the same as the previous lab, you can save any plots produced by your experiments for easier comparison across different runs.

Answer the questions by configuring the `Lab3ExercisesRunner` and `Lab3TestFrameConfig` classes as appropriate, running the algorithms, and analysing the resulting outputs. You may want to keep a copy of the results presented after running each of those experiments.

Note that feedback for open textbox questions will be provided no later than the next in-person lab and will address common themes at the cohort level. If you wish to discuss individual answers, please ask one of us during one of the lab sessions.

ADDITIONAL TIPS / FAQ

Where do I find the mutation and local search heuristics?

Both mutation and local search heuristics are created within `Lab3ExerciseRunner` and are passed into the constructor of the `IteratedLocalSearch` Class. They are then stored as member variables named `oMutationHeuristic` and `oLocalSearchHeuristic` respectively.

How do I use different/multiple heuristics within a single search method?

A heuristic h is applied to a solution by calling `h.applyHeuristic(oProblem, iSolutionIndex)`. Remember `SATHeuristics` within this framework modify the solution in the `CURRENT_SOLUTION_INDEX` **only**. Hence, multiple `applyHeuristic` calls can be made one after another with each heuristic receiving the candidate solution generated by the previous heuristic.

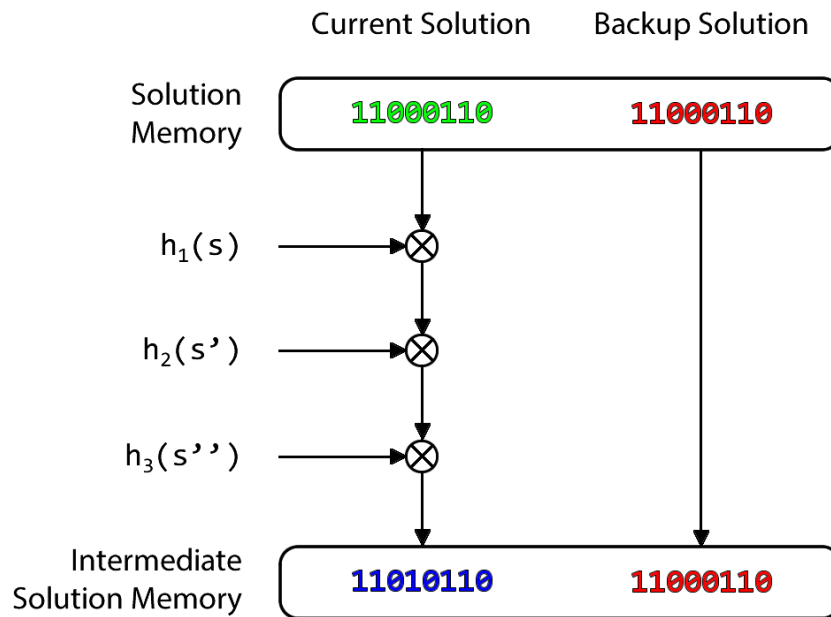


Figure 2 - Illustration of chaining multiple heuristics in the COMP2001 Framework.

How do I: perform a bit flip; get the number of variables; evaluate the solution; etc.

See the COMP2001 Framework API for framework specific questions on Moodle!